



كلية الهندسة المعلوماتية – السنة الرابعة – قسم الذكاء الصناعي - 2026

تقرير البرمجة المتوازية

High-Performance E-Commerce Backend Engine

الطلاب المشاركون:

عاطف أسامة شعبان – عمر محمد القاسم

وجد بشار غريب – أمجد علي عيسى – حلا ياسين زين

مقدمة:

يهدف هذا المشروع إلى بناء نظام تجارة إلكترونية عالي الأداء قادر على التعامل مع عدد كبير من الطلبات المتزامنة بكفاءة واستقرار، مع تطبيق مفاهيم البرمجة المتوازية والأنظمة الموزعة.

يركز المشروع على حماية البيانات من التضارب، إدارة الموارد، تنفيذ المعالجة غير المتزامنة، معالجة البيانات على دفعات، توزيع الأحمال، وضمان سلامة المعاملات والعمليات المتزامنة تحت الضغط العالي.

تم تطوير المشروع باستخدام Django و Celery و Threading بهدف بناء نظام مستقر وسريع وقابل للتوسع.

الطلبات:

الطلب الأول: حماية البيانات المشتركة من التضارب
(Concurrent Access & Data Integrity)

الحل المستخدم:

تم استخدام مفاهيم:

• Pessimistic Locking

• Database Row Locking

وذلك من خلال :

```
transaction.atomic()
```

تقوم هذه الدالة بجعل جميع العمليات داخلها تنفذ كعملية واحدة، فإذا حدث خطأ يتم التراجع عن جميع التعديلات للحفاظ على سلامة البيانات.

```
select_for_update()
```

تقوم بقفل الصف داخل قاعدة البيانات أثناء التعديل عليه، بحيث لا يستطيع أي مستخدم آخر تعديل نفس المنتج حتى انتهاء العملية الحالية.

سبب استخدامه:

عند قيام مستخدم بشراء منتج، يتم قفل الصف الخاص بالمنتج داخل قاعدة البيانات حتى انتهاء العملية، مما يمنع أي مستخدم آخر من تعديل نفس المنتج في نفس اللحظة.

المشكلة التي تم حلها:

بدون استخدام القفل قد تحدث مشكلة Race Condition كما يلي:

- المستخدم A يقرأ أن الكمية = 1

- المستخدم B يقرأ أن الكمية = 1

- كلاهما يقوم بالشراء.

- تصبح الكمية بالسالب.

أما باستخدام القفل فإن النظام يسمح لمستخدم واحد فقط بتعديل الكمية في نفس الوقت، وبالتالي يتم منع تضارب البيانات.

حل إضافي مستخدم:

تم أيضاً استخدام الدالة **F()**:

تسمح بتنفيذ العمليات الحسابية مباشرة داخل قاعدة البيانات بدون الحاجة إلى قراءة القيمة أولاً ثم تعديلها داخل الكود.

الهدف من ذلك:

تنفيذ تحديث مباشر ومن داخل قاعدة البيانات باستخدام

```
stock_quantity = F('stock_quantity') - 1
```

بدلاً من قراءة القيمة ثم تعديلها يدوياً.

الفائدة:

- تحديث Atomic وآمن.
- أسرع تحت الضغط العالي.
- تقليل احتمالية حدوث Race Condition.
- تحسين الأداء.

الطلب الثاني: إدارة الموارد والتحكم بالسعة (Resource Management & Capacity Control)

الحل المستخدم:

Semaphore(10)

تستخدم لتحديد عدد العمليات التي يسمح للنظام بتنفيذها في نفس الوقت.

سبب استخدامه:

ال Semaphore يسمح بالتحكم بعدد العمليات المتزامنة التي يستطيع النظام تنفيذها في نفس الوقت.

آلية العمل:

- يسمح فقط لـ 10 عمليات شراء متزامنة.
- أي طلبات إضافية يتم انتظارها أو رفضها.

الفائدة:

بدون استخدام Semaphore:

قد تصل مئات الطلبات دفعة واحدة.

- يزداد استهلاك CPU و RAM.

- احتمال انهيار السيرفر .

أما باستخدامه:

- يبقى السيرفر مستقرًا.

- يتم التحكم بالحمل.

- تحقيق Capacity Control.

تم استخدام:

`ThreadPoolExecutor()`

تقوم بإنشاء مجموعة ثابتة من الـ Threads وإعادة استخدامها لتنفيذ المهام بالخلفية بدلاً من إنشاء Thread جديد لكل عملية.

السبب:

منع إنشاء عدد غير محدود من ال Threads داخل النظام.

آلية العمل:

- يسمح فقط بتشغيل 5 Threads بالخلفية.
- يتم إعادة استخدام ال Threads الموجودة.

الفائدة:

- تحسين الأداء.
- تقليل استهلاك الذاكرة.
- زيادة استقرار النظام.
- تحسين إدارة الموارد.

الطلب الثالث: المعالجة غير المتزامنة

(Asynchronous Processing)

تم استخدام:

• Celery Tasks

• Background Threads

تنفيذ Celery مثل `send_verification_email.delay(user.email)`

شرح الدالة المستخدمة:

`delay()`

تقوم بإرسال المهمة إلى Celery Queue ليتم تنفيذها بالخلفية بواسطة Worker بدون إيقاف الطلب الرئيسي.

سبب استخدامه:

بدلاً من جعل المستخدم ينتظر إرسال البريد الإلكتروني أثناء تنفيذ الطلب:

- يتم إنهاء الطلب مباشرة.
- إرسال المهمة إلى Queue.
- يقوم Worker بمعالجتها بالخلفية.

الفائدة:

- تقليل زمن الاستجابة.
- تحسين تجربة المستخدم.
- تقليل الضغط على ال Request الرئيسي.

تم استخدام Thread Executor أيضا من خلال دالة:

`submit()`

تقوم بإرسال مهمة ليتم تنفيذها داخل Thread بالخلفية بشكل غير متزامن.

الهدف:

تشغيل مهام Background محلية باستخدام Threads.

الفائدة:

- تنفيذ المهام بشكل غير متزامن.
- عدم تعطيل المستخدم أثناء تنفيذ العمليات الطويلة.
- تحسين سرعة النظام.

الطلب الرابع: معالجة البيانات على دفعات

(Batch Processing)

الحل المستخدم:

تم تنفيذ ذلك داخل `seller_sales_analytics()` حيث تم تقسيم البيانات إلى دفعات صغيرة باستخدام `chunks` ثم معالجة كل دفعة بشكل مستقل.

شرح الدالة المستخدمة:

`seller_sales_analytics()`

تقوم بجلب بيانات المبيعات ثم تقسيمها إلى مجموعات صغيرة وإرسال كل مجموعة لمعالجتها بشكل مستقل.

سبب استخدامه:

بدلاً من معالجة الالف السجلات دفعة واحدة، تم تقسيمها إلى مجموعات صغيرة لتحسين الأداء.

الفائدة:

- تنفيذ Parallel Processing.
- سرعة اعلى في المعالجة.
- استهلاك ذاكرة أقل.

- تحسين قابلية التوسع.
- تقليل الضغط على قاعدة البيانات.

الطلب الخامس: توزيع الأحمال

(Load Distribution)

الدوال المستخدمة:

`group()`

تقوم بتشغيل عدة مهام Celery بالتوازي في نفس الوقت.

`process_checkout_item()`

تقوم بمعالجة منتج واحد داخل الطلب مثل تحديث الكمية وإنشاء OrderItem.

`checkoutLoadDistribution()`

تقوم بتقسيم الطلب إلى عدة مهام مستقلة ثم توزيعها على الـ Workers.

سبب استخدامه:

يقوم Celery Group بتوزيع المهام على عدة Workers بالتوازي.

آلية العمل:

- كل منتج داخل الطلب يتم معالجته بشكل مستقل
- عدة Workers يعملون بنفس الوقت
- توزيع الحمل على أكثر من عملية تنفيذ

الفائدة:

- تسريع تنفيذ الطلبات الكبيرة
- تحسين استجابة النظام
- محاكاة مفهوم Load Balancing
- الاستفادة من المعالجة المتوازية

تجارب التشغيل:

التجارب باستخدام locust

تم الوصول إلى request 12,000 دون مواجهة أي fail.

القدرة القصوى: 30,000.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/token/	100	0	26	43	80	26.98	4	80	497	0	0
POST	/cart/add/	5949	0	11	30	66	14.12	7	558	27	65.1	0
POST	/checkout_distribution/	5948	0	14	42	170	20.94	9	524	102	65	0
POST	/register/	100	0	720	1500	1600	755.18	143	1586	51	0	0
	Aggregated	12097	0	12	41	270	23.71	4	1586	67.96	130.1	0

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/token/	100	0	22	45	55	24.34	10	55
POST	/cart/add/	2584	0	11	40	71	15.1	7	337
POST	/checkout_distribution/	2583	0	14	56	140	21.61	8	277
POST	/register/	100	0	130	290	310	142.66	84	313
	Aggregated	5367	0	13	60	150	20.78	7	337



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/token/	200	0	29	69	110	30.83	3	116	497	0	0
POST	/cart/add/	6491	14	91	450	850	143.47	7	2546	612.68	79.8	1.4
POST	/checkout_distribution/	6383	7	210	950	2000	316.21	8	2970	399.81	80.9	0.7
POST	/register/	200	0	190	480	580	220.24	16	594	51	0	0
	Aggregated	13274	21	120	780	1400	225.99	3	2970	500.11	160.7	2.1

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/token/	200	0	29	69	110	30.83	3	116	497	0	0
POST	/cart/add/	15145	465	170	1400	6500	439.98	7	14559	8365.95	93.4	1.9
POST	/checkout_distribution/	15139	354	400	3000	6400	762.96	8	14576	6456.95	99.8	1.1
POST	/register/	200	0	190	480	580	220.24	16	594	51	0	0
	Aggregated	30684	819	230	2300	6500	595.23	3	14576	7318.59	193.2	3

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/token/	200	0	22	55	65	24.44	6	65	497	0	0
POST	/cart/add/	2249	0	11	40	90	16.23	6	210	27	69.5	0
POST	/checkout_distribution/	2244	0	14	71	190	24.45	9	381	102	69.5	0
POST	/register/	200	0	190	450	600	208.59	72	612	68.07	0	0
GET	/seller_sales?period=day	337	0	8	27	63	10.58	4	75	108	9.8	0
GET	/seller_sales?period=month	343	0	7	34	85	11.59	4	148	108	9.5	0
GET	/seller_sales?period=year	363	0	7	26	67	10.31	4	78	108	9.9	0
	Aggregated	5936	0	12	90	240	25.15	4	612	86.8	168.2	0